Segunda Fase Proyecto

Proyecto:

Se desarrollará una aplicación web utilizando **ASP.NET Core** como plataforma principal de desarrollo, incorporando **Entity Framework Core** para la gestión de datos y una base de datos seleccionada por el equipo según las necesidades del proyecto. La solución deberá integrar una funcionalidad basada en **Inteligencia Artificial (IA)** que aporte valor al problema planteado.

La implementación utilizará **Docker** para la contenerización de la aplicación, garantizando la portabilidad, mantenibilidad y facilidad de despliegue en distintos entornos. Adicionalmente, los equipos deberán investigar y proponer una estrategia de despliegue utilizando **Microsoft Azure** como plataforma de servicios en la nube.

El proyecto se llevará a cabo en dos fases:

2. **Fase I:** Diseño, planificación y documentación de la solución, incluyendo análisis del problema, arquitectura del sistema, diseño de base de datos, prototipos UX/UI, modelo de negocio y planificación del desarrollo.
3. **Fase II:** Desarrollo, implementación e integración de la solución, incorporando la funcionalidad de Inteligencia Artificial, la contenerización con Docker, la integración de los distintos componentes del sistema y la validación del funcionamiento completo de la aplicación.

Instrucciones Generales:

- El proyecto se llevará a cabo en grupos de cinco personas. Se debe seleccionar un coordinador y este será quien realice las entregas. El lenguaje para desarrollo **Backend y Frontend** deben de usar **C# .NET** , **asp.net core**
- Una vez formados los equipos de trabajo no se pueden disolver ni cambiar de grupos. **Si realiza esto su nota será 0.0**
- La entrega del documento se realizará a través del aula virtual.
- No habrá defensa en la primera entrega, solo entrega del documento en pdf.
- Se anulará la nota a ambos grupos en caso de que se presenten trabajos similares.
- Todas las consultas serán atendidas a través de los canales de comunicación definidos al inicio del ciclo, en un horario de 8:00 am a 4:00 pm.

FECHA DE ENTREGA: 18 de octubre de 2026

IMPORTANTE:

- *Cada punto no presentado cómo se solicita en las indicaciones generales, se le restará 0.20 puntos a su calificación total de la actividad.*
- *Quien no asistan a la presentación del proyecto no tendrán derecho a calificación en toda la actividad y se le pondrá como nota 0.*

Fase II: Desarrollo, Implementación e Integración del Prototipo Funcional

Enunciado

En esta fase, el equipo deberá desarrollar, integrar e implementar la versión funcional del sistema propuesto durante la Fase I, incorporando los componentes tecnológicos definidos en la Investigación Aplicada II.

La solución deberá estar desarrollada utilizando **ASP.NET Core**, **Entity Framework Core** y una base de datos relacional o no relacional, siguiendo una arquitectura modular y escalable.

Asimismo, el proyecto deberá considerar la incorporación de tecnologías modernas del ecosistema Microsoft, incluyendo:

- Inteligencia Artificial (IA).
- Docker para la contenerización de la aplicación.
- Microsoft Azure como alternativa de despliegue en la nube.
- APIs y servicios web necesarios para la integración del sistema.

El prototipo deberá demostrar el funcionamiento de los módulos principales definidos en la fase de planificación, así como la integración de la característica inteligente propuesta durante la Investigación Aplicada II.

Se espera que la entrega incluya el sistema funcional, documentación técnica, evidencias de pruebas, video demostrativo y repositorio actualizado.

Productos Esperados

1. Aplicación Web Funcional

El sistema deberá estar completamente operativo y ejecutable.
Debe incluir:

- Navegación entre módulos.
- Interfaces funcionales.
- Formularios y validaciones.
- Gestión de usuarios (si aplica).
- Operaciones CRUD.
- Consumo de APIs o servicios externos.
- Acceso a datos mediante Entity Framework Core.

La aplicación deberá reflejar los diseños UX/UI definidos en la Fase I.

2. Backend Operativo

Implementar una solución basada en ASP.NET Core.
Debe contemplar:

- Arquitectura en capas.
- Operaciones CRUD.
- Gestión de base de datos.
- Entity Framework Core.
- Validaciones.
- Manejo de excepciones.
- Seguridad básica.

Opcionalmente:

- Web API REST.
- JWT.
- Roles y permisos.

3. Integración de Inteligencia Artificial

El equipo deberá implementar al menos una funcionalidad basada en IA propuesta durante la Investigación Aplicada II.
Ejemplos:

- Chatbot inteligente.
- Asistente virtual.

- Generación automática de reportes.
- Sistema de recomendaciones.
- Clasificación automática de información.
- Resumen automático de datos.
- Búsqueda inteligente mediante lenguaje natural.

Se deberá evidenciar:

- Datos de entrada.
- Procesamiento realizado.
- Resultados generados.
- Beneficio aportado al sistema.

4. Docker y Despliegue

La solución deberá incluir la contenerización de la aplicación mediante Docker.
Se recomienda incluir:

- Dockerfile.
- docker-compose.yml.
- Configuración de variables de entorno.

Además, deberán presentar una propuesta de despliegue utilizando Microsoft Azure, indicando:

- Servicio Azure seleccionado.
- Beneficios.
- Costos aproximados.
- Arquitectura de despliegue.

No es obligatorio desplegar el proyecto en Azure, pero sí documentar su posible implementación.

5. Integración General del Sistema

Se deberá demostrar:

- Comunicación entre módulos.
- Integración entre frontend y backend.
- Integración con base de datos.
- Integración de la funcionalidad de IA.
- Funcionamiento completo de la solución.

6. Pruebas de Funcionamiento

Realizar pruebas que evidencien:

- Funcionamiento del sistema.
- Correcta ejecución de casos de uso.
- Integración de componentes.
- Funcionamiento de la característica de IA.
- Ejecución dentro del contenedor Docker.
- Debe incluir evidencia de las pruebas realizadas.

7. Documentación

Manual de Instalación

Debe incluir:

- Clonación del repositorio.
- Instalación de dependencias.
- Configuración de variables de entorno.
- Ejecución del proyecto.
- Ejecución mediante Docker.

Manual de Usuario

Debe incluir:

- Acceso al sistema.
- Descripción de funcionalidades.
- Uso de la característica de IA.

Manual Técnico

Debe incluir:

- Arquitectura del sistema.
- Tecnologías utilizadas.
- Estructura del proyecto.
- Configuraciones realizadas.
- Integración de IA.
- Configuración Docker.

8. Video de Evidencia

Video de entre 5 y 10 minutos donde se muestre:

- Presentación breve del problema.
- Arquitectura desarrollada.
- Funcionamiento del sistema.
- Operaciones CRUD.
- Funcionalidad de IA implementada.
- Ejecución del proyecto mediante Docker.
- Resultados obtenidos.
- Se deberá evidenciar la participación de todos los integrantes.

9. Repositorio del Proyecto

El repositorio deberá contener:

Estructura organizada

- Plain Text
- /src
- /docs
- /database
- /docker

Además:

- Código fuente documentado.
- README actualizado.
- Archivo .env.example.
- Historial de commits.
- Evidencias de trabajo colaborativo.

Entregables

- Sistema funcional desarrollado en ASP.NET Core.
- Base de datos implementada.
- Funcionalidad de Inteligencia Artificial integrada.
- Archivos Docker (Dockerfile y/o docker-compose).
- Manual de usuario.
- Manual técnico.
- Evidencias de pruebas.
- Video demostrativo.

- Repositorio Git actualizado.
- Documento de avance técnico actualizado.

Rúbrica de Evaluación Grupal (50%)

Criterio	Excelente (5)	Bueno (4)	Aceptable (3)	Insuficiente (2)
1. Aplicación Web Funcional	La aplicación funciona completamente, cumple todos los requerimientos y presenta una experiencia de usuario fluida.	La aplicación es funcional con pequeños detalles por mejorar.	La aplicación presenta funcionalidades básicas y algunas limitaciones.	La aplicación presenta errores importantes o no cumple los requerimientos mínimos.
2. Backend y lógica de negocio	Backend completamente funcional, organizado y con correcta implementación de reglas de negocio.	Backend funcional con pequeños inconvenientes.	Backend parcialmente implementado o con funcionalidades limitadas.	Backend incompleto o con fallos críticos.
3. Gestión de Base de Datos y Entity Framework Core	Modelo de datos correctamente implementado, con relaciones, validaciones y acceso eficiente a la información.	Implementación adecuada con pequeños problemas de diseño.	Implementación básica con inconsistencias.	Implementación deficiente o inexistente.
4. Operaciones CRUD	Todas las operaciones CRUD funcionan correctamente y están integradas al sistema.	CRUD funcional con pequeños errores.	CRUD parcialmente implementado.	CRUD incompleto o no funcional.
5. Integración de Inteligencia Artificial	La funcionalidad de IA está completamente integrada,	La funcionalidad de IA es funcional y	La integración de IA es básica o limitada.	No implementa IA o su funcionamiento es incorrecto.

	aporta valor al sistema y demuestra un uso adecuado de la tecnología.	aporta valor al proyecto.		
6. Integración general del sistema	Todos los módulos funcionan de forma integrada y estable.	La integración es adecuada con pequeños inconvenientes.	Existen problemas de integración entre algunos componentes.	Los módulos no funcionan de forma integrada.
7. Docker y despliegue	La aplicación está correctamente contenerizada y documentada, demostrando dominio de Docker.	Docker implementado correctamente con pequeños detalles faltantes.	Docker implementado parcialmente o con errores menores.	No presenta contenerización funcional.
8. Diseño UX/UI e interfaz de usuario	Interfaces completas, atractivas, intuitivas y consistentes con el diseño propuesto.	Diseño funcional y visualmente adecuado.	Diseño básico con limitaciones de navegación o usabilidad.	Interfaces incompletas o poco funcionales.
9. Pruebas y validación	Presenta evidencias completas de pruebas funcionales, integración y validación de resultados.	Presenta pruebas adecuadas del funcionamiento.	Presenta pruebas parciales o poco documentadas.	No presenta evidencias suficientes de pruebas.
10. Documentación técnica	Manuales completos, claros y correctamente estructurados.	Documentación adecuada con pequeños detalles faltantes.	Documentación básica o incompleta.	Documentación insuficiente o inexistente.
11. Repositorio Git	Repositorio organizado, documentado y con evidencia constante de trabajo colaborativo.	Repositorio adecuado con pocos detalles faltantes.	Repositorio básico con poca documentación.	Repositorio incompleto o desorganizado.

12. Presentación Final (Pitch Técnico)	Presentación profesional, clara y bien estructurada. Explica el problema, la solución, la arquitectura, las tecnologías utilizadas, los retos enfrentados y las lecciones aprendidas. Todos los integrantes participan activamente y demuestran dominio del proyecto.	Presentación clara y organizada, con participación de la mayoría de los integrantes y dominio adecuado del proyecto.	Presentación básica, con explicaciones generales y participación limitada del equipo.	Presentación desorganizada, incompleta o con escasa participación y conocimiento del proyecto.

Penalización por Entrega Tardía para parte grupal

Días de Retraso	Penalización Aplicada
Entregado a tiempo	Sin penalización (0%)
1 día de retraso	-20% sobre la calificación final
2 días de retraso	-30% sobre la calificación final
3 días de retraso	-50% sobre la calificación final
Más de 3 días	No se acepta y se califica con 0 (salvo justificación válida)

Defensa Individual (50%)

La calificación correspondiente a la Defensa Individual se distribuye de la siguiente manera:

- **Asistencia a clases (10%)**
  Se evaluará la participación y el cumplimiento de la asistencia a lo largo del curso, considerando la presencia constante y activa del estudiante en las sesiones programadas.
- **Test sobre los contenidos del ciclo (30%)**
  Corresponde a la calificación obtenida en la evaluación integral del contenido del ciclo.
- **Coevaluación grupal (10%)**
  Se valorará el desempeño individual dentro del equipo de trabajo, considerando la percepción de sus compañeros respecto a su contribución, responsabilidad y colaboración durante el desarrollo del proyecto.